

C3 - Intelligence pour la robotique: TP7 – Apprentissage supervisé

Justin Ferdinand

I EXERCICES DE CLASSIFICATION

Dans cette partie, j'explore les résultats donnés par les expérimentations réalisées dans le notebook TP7_Classification.ipynb. Ce fichier contient en détail ces réalisations.

I.A CLASSIFICATION AVEC PERCEPTRON

Le Perceptron est un classifieur linéaire capable d'apprendre les fonctions logiques linéairement séparables. Les expériences suivantes démontrent cette capacité et ses limites.

I.A.1 QUESTION 1 - OPÉRATEUR OU

Le Perceptron a atteint une précision de 100% (accuracy = 1.0) pour l'opérateur OU, avec les prédictions : [0, 1, 1, 1], correspondant parfaitement aux valeurs attendues.

Cela vient du fait que l'opérateur OU est linéairement séparable dans le plan d'entrée. Une seule droite peut séparer les points de classe 0 (0,0) des points de classe 1 ((0,1), (1,0), (1,1)).

I.A.2 QUESTION 2 - OPÉRATEUR ET

Le Perceptron a également atteint une précision de 100% pour l'opérateur ET, avec les prédictions : [0, 0, 0, 1], correspondant aux valeurs attendues. L'opérateur ET est aussi linéairement séparable.

I.A.3 QUESTION 3 - OPÉRATEUR XOR

Le Perceptron n'a atteint qu'une précision de 50% pour l'opérateur XOR, avec les prédictions :

[0, 0, 0, 0], alors que les valeurs attendues sont [0, 1, 1, 0].

Cela vient du fait qu'il n'est pas possible de séparer les classes du XOR avec une seule ligne droite. Les points (0,1) et (1,0) sont de classe 1, tandis que (0,0) et (1,1) sont de classe 0, ce qui forme un motif en damier.

I.B MLP POUR XOR

Pour résoudre le problème non-linéaire du XOR, j'utilise un perceptron multicouche (MLP) avec une couche cachée.

I.B.1 QUESTIONS 4 & 5 - CONFIGURATION ET RÉSULTATS

Configuration du MLP :

- Activation : tanh
- Couches cachées : 1 couche de 2 neurones
- Max iterations : 10000
- Solver : lbfgs

Un premier entraînement a produit les prédictions : [0, 0, 1, 1], obtenant un score de 50%. Cependant, ce résultat correspond à une convergence partielle.

I.B.2 TEST DE STABILITÉ SUR 10 ESSAIS

Scores individuels : [1.0, 1.0, 0.75, 1.0, 1.0, 1.0, 0.5, 0.75, 0.75, 1.0] donc nombre de convergences réussies : 6 fois sur 10 (60%)

Le réseau peut converger vers une solution parfaite, mais l'entraînement est instable. L'initialisation aléatoire des poids ("random_state=None") entraîne une variabilité importante. Certains essais convergent complète-

ment (score = 1.0), tandis que d'autres se bloquent à des solutions limitées.

I.C EXTRACTION DES POIDS

I.C.1 QUESTION 6 - POIDS D'UN MODÈLE CONVERGENT

Pour un entraînement réussi (score = 1.0), les poids extraits du réseau MLP sont :

Couche cachée (Entrée → Cachée) :

- $w_1(x_1 \rightarrow h_1) = 3.4124$
- $w_2(x_1 \rightarrow h_2) = -3.5106$
- $w_3(x_2 \rightarrow h_1) = -3.9323$
- $w_4(x_2 \rightarrow h_2) = 3.5706$
- $b_1(\text{biais } h_1) = 2.4247$
- $b_2(\text{biais } h_2) = 1.8764$

Couche de sortie (Cachée → Sortie) :

- $w_5(h_1 \rightarrow \text{out}) = -8.6213$
- $w_6(h_2 \rightarrow \text{out}) = -8.3504$
- $b_3(\text{biais out}) = 8.7334$

I.C.2 ANALYSE DE LA DÉCISION

Les poids élevés en valeur absolue reflètent l'utilisation de la fonction d'activation tanh qui doit créer des frontières de décision nettes.

Pour le neurone h_1 , le poids positif important pour x_1 (3.4124) et le poids négatif pour x_2 (-3.9323) indiquent que h_1 s'active fortement lorsque x_1 est élevé et x_2 est faible, créant une partition du plan d'entrée. C'est le cas inverse pour le neurone h_2 .

Un biais de sortie fort et positif (8.7334) compense les poids négatifs de h_1 et h_2 pour créer la logique XOR.

I.D XOR EN RÉGRESSION

Pour résoudre XOR en utilisant une approche de régression plutôt que de classification :

Configuration :

- Modèle : MLPRegressor
- Architecture : 2 couches cachées de 4 neurones
- Activation : tanh

Résultats :

Prédictions (arrondies) : $[\approx 0.00, \approx 1.00, \approx 1.00, \approx 0.00]$

Score R^2 : 0.9999996073410653 (presque parfait)

Convergence en 126 itérations.

Je constate donc que l'approche par régression est plus stable et converge vers une solution quasi-parfaite. L'architecture plus profonde (deux couches cachées) facilite l'apprentissage de ce problème non-linéaire sans l'instabilité observée en classification.

II EXERCICE D'APPLICATION ROBOTIQUE

Dans cette partie, j'ai appliqué les concepts d'apprentissage supervisé à un problème de contrôle de robot en utilisant un réseau de neurones artificiels pour apprendre à éviter les obstacles.

II.A ENREGISTREMENT DES DONNÉES

J'ai utilisé le script `controleur_dataset_gen.py` pour enregistrer les données de capteurs et de commandes de vitesse du robot pendant une session de contrôle manuel. Les données comprennent les lectures des capteurs de proximité, les scans LIDAR, les commandes de vitesse manuelles, et les prédictions du réseau de neurones. J'ai ainsi réalisé plusieurs tours du circuit dans les deux sens avec des commandes manuelles.

II.B PROTOCOLES DE TEST ET FUSION PROGRESSIVE DE CAPTEURS

II.B.1 TEST 1 : CAPTEURS DE PROXIMITÉ UNIQUEMENT

La première expérience réalisée a consisté à entraîner le réseau exclusivement sur les données des capteurs de proximité pour tester que notre

modèle soit lisible et que les sorties moteurs soient compréhensibles par le robot.

Configuration du modèle :

- Architecture : 2 couches cachées de 64 neurones chacune
- Activation : ReLU
- Solver : Adam (taux d'apprentissage = $1e-3$)

Capteurs utilisés : 7 capteurs de proximité du Thymio (normalisés sur $[0,1]$)

Ce test n'a pas été concluant avec un score R^2 de 0.55, ce qui est insuffisant pour un contrôle fiable du robot. Cependant, cela m'a permis de comprendre comment exporter le modèle après son entraînement et de vérifier que les prédictions sont dans la bonne plage pour les commandes moteurs.

II.B.2 TEST 2 : FUSION LiDAR + CAMÉRA

La deuxième expérience combine deux capteurs plus riches en information :

Capteurs utilisés :

- LiDAR
- Caméra

Configuration du modèle :

- Architecture : 2 couches cachées (64, 64)
- Activation : ReLU

Ce modèle a obtenu un score R^2 de 0.92, ce qui est une amélioration significative par rapport au test précédent. L'ajout du LiDAR et de la caméra a permis au réseau d'apprendre des représentations plus complexes de l'environnement, améliorant ainsi la précision des prédictions de vitesse. En test, Thymio était capable de suivre les murs mais avait parfois des difficultés dans les virages serrés où l'absence de capteur de proximité limitait la réactivité.

La capture vidéo de ce test s'intitule `lidarANDcamera_fail.mp4` et montre les limites

de cette configuration, notamment dans les virages où le robot a tendance à couper les angles.

II.B.3 TEST 3 : FUSION COMPLÈTE (LiDAR + CAMÉRA + PROXIMITÉ)

La dernière approche intègre tous les capteurs disponibles pour une perception maximale :

Capteurs utilisés :

- LiDAR
- Caméra
- Proximité

Configuration du modèle :

- Architecture : 2 couches cachées (64, 64)
- Activation : ReLU
- Solver : Adam avec `early_stopping=True`
- Validation : 10% des données pour éviter le surapprentissage
- Époque d'arrêt : 30 itérations sans amélioration

Pipeline de normalisation :

- | | |
|---|--|
| 1 | 1. Conversion LiDAR 2D → normalisation $[0,1]$ |
| 2 | 2. Extraction caméra HSV → normalisation $[0,1]$ |
| 3 | 3. Normalisation proximité par L_1 (identique collecte de données) |
| 4 | 4. StandardScaler sur ensemble d'entraînement |
| 5 | 5. Prédiction vitesses moteurs $[-1, 1]$ |

Résultats d'évaluation :

- Score R^2 (Test) : 0.9654

Ce test avec la fusion de toutes les données a été le plus concluant, il a le meilleur score R^2 et Thymio est plus réactif, capable de suivre les murs et virages. J'observe qu'il fonctionne tout aussi bien dans un sens comme dans l'autre.

Cependant, j'observe une limite que j'avais déjà rencontrée lors d'un stage sur les bras robotiques intelligents par modèle VLA (Vision Langage Action): Le robot est très peu capable de faire de la reprise à l'erreur.

Par exemple, en mettant le robot face à un mur, j'observe que son entraînement ne lui a pas permis d'apprendre à reculer ou à tourner sur place pour se dégager. Il continue à avancer et finit par heurter le mur, ce qui montre que le modèle n'a pas appris de stratégie de contournement ou de correction d'erreur. Il a simplement appris à sortir une commande moteur en fonction des données capteurs, mais sans une réelle compréhension de son environnement.

Pour améliorer cela, il faudrait enrichir le jeu de données par des scénarios d'échec pour que le modèle puisse apprendre à réagir face à ces situations. Il s'agissait là d'une des limites trouvée lors de mon stage où toutes les situations ne pouvaient pas être couvertes par les données d'entraînement, ce qui limitait la capacité du robot à faire face à des cas imprévus par l'ingénieur.

La capture vidéo de ce test s'intitule `lidarANDcameraANDprox.mp4` et montre une amélioration significative de la capacité du robot à suivre les murs et à négocier les virages.

II.C ENTRAÎNEMENT DU RÉSEAU DE NEURONES

Les modèles sont entraînés avec le notebook `TP7_read_data.ipynb` qui :

1. Charge le dataset HDF5 contenant les 95 463 échantillons sensoriels
2. Sélectionne les features selon le mode choisi
3. Applique un pipeline de normalisation (StandardScaler)
4. Entraîne l'architecture MLP avec validation croisée
5. Sauvegarde le modèle si $R^2 > 0.94$

II.D INTÉGRATION DU MODÈLE DANS WEBOTS

Procédure de chargement du modèle :

1. Initialisation Webots
2. Accès aux périphériques :
 - Moteurs gauche/droit (SetVelocity)
 - Caméra RGB
 - LiDAR
 - 7 capteurs de proximité normalisés
3. Chargement du modèle :
 - Path: Dans le contrôleur webots
 - Format: Pipeline (StandardScaler + MLPRegressor)
4. Boucle de contrôle :
 - Lecture capteurs bruts
 - Normalisation identique à l'entraînement
 - Passage au pipeline sklearn
 - Extraction prédictions [left_velocity, right_velocity]
 - Envoi commandes moteurs

Pour conclure, ce TP m'a permis de mettre en pratique les concepts d'apprentissage supervisé dans un contexte de contrôle de robot. J'ai pu expérimenter différentes architectures de réseaux de neurones et observer l'impact de la fusion de capteurs sur les performances du modèle. J'ai également constaté les limites de l'approche, notamment en termes de capacité à faire face à des situations imprévues, ce qui souligne l'importance de la diversité des données d'entraînement pour les systèmes robotiques intelligents.